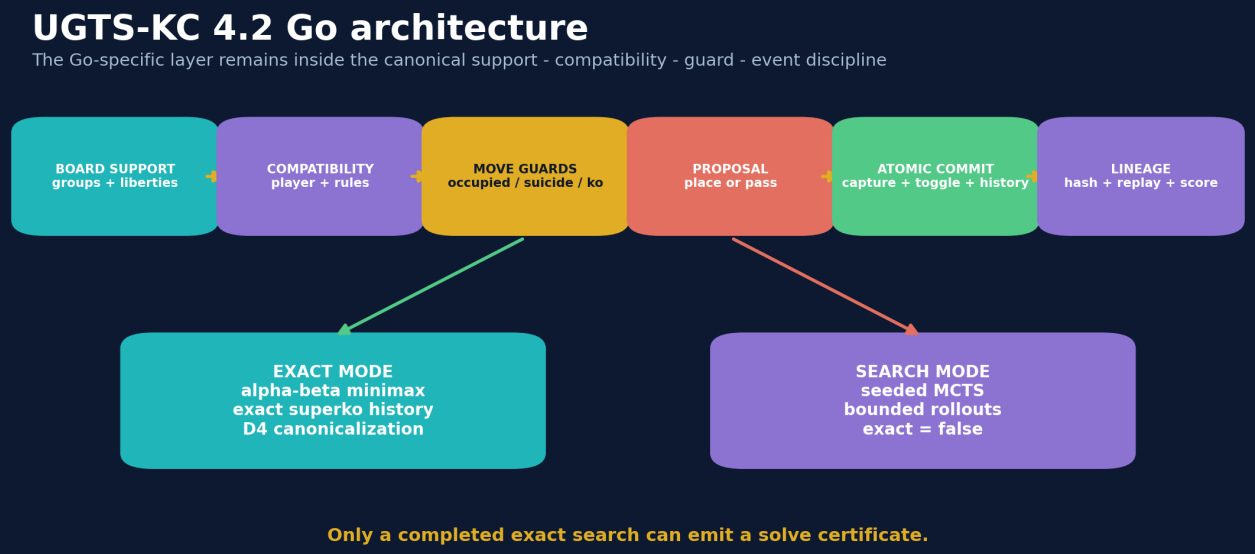
UGTS-KC 4.2 GO

Exact Finite Solving, Proof Certificates, Scalable Search and Self-Contained Runtime

An additive board-game application profile over the supplied UGTS substrate line



Prepared for	Tom Klootwijk
Version	4.2.0
Project schema	ugts-go-state-4.2 / ugts-go-solve-certificate-4.2
Document date	29 August 2026
Status	Executable reference package with bounded exact evidence

Executive Summary

FORMAL DECISION

The requested objective is admitted as a proof-oriented Go profile: model every move as a verified UGTS event, make superko history part of exact state identity, solve bounded finite positions by exhaustive minimax, and separate exact certificates from seeded search estimates. The result is a coherent implementation and release package. It does **not** claim a game-theoretic solution of unrestricted standard 19x19 Go.

VERIFIED RELEASE RESULT

The package contains 60 new cataloged mechanisms, M390–M449; 52 dependency-free tests with 52/52 passing; zero JSON Schema errors; a verified exact certificate for the empty 2x2 game at 0.5 komi; a deterministic 9x9 MCTS evidence run; a self-contained HTML board; SGF import/export; a wheel build; and a complete SHA-256 manifest.
Under the declared exact profile (area scoring, no suicide, positional superko, 0.5 komi), the empty 2x2 game has exact minimax value **Black +0.5**. The certificate hash is:

`eaec58fd1dbf2ccd0b81981750385b5112b858229f599c0baaa0c4fd17f4be0d`

EVIDENCE / CLAIM BOUNDARY

"Solve" has three different meanings that are kept separate:

- **Rules solved**: every legal move, capture, pass, repetition and score transition is deterministic and testable.
- **Position solved**: an exhaustive search completed for one fully serialized finite state and rules profile.
- **Game solved**: the initial standard board is proven under a complete rules profile. This release establishes the first two for bounded fixtures, not the third for 19x19.

Table 1: Release deliverables

Deliverable	Contents	Status
PDF technical report	Architecture, rules, solver, certificate, validation and catalog	Delivered
Python package	Deterministic rules, exact solver, MCTS, SGF, CLI, web builder	Tested
Exact evidence	1x1 and 2x2 solved fixtures plus rerunnable certificate	Verified
Search evidence	Seeded 9x9 MCTS with explicit non-proof marker	Verified estimate
Self-contained browser	One HTML file; no CDN or network asset	Built
Machine-readable spec	JSON Schema, source map, M390–M449 CSV/JSON	Validated

Contents

1. Source Basis and Integration Discipline

The supplied files do not contain a Go solver. They do contain a consistent architecture that can be applied without replacing their evidence boundaries. The Go rules, scoring and search algorithms in this release are therefore visibly labeled *engineering-derived*, while the surrounding state/event discipline is inherited.

Table 2: Suitable supplied substrate versions and their use

Source	Used mechanisms	Treatment
UGTS-KC 2.0	Query-first state, topology descriptors, support/compatibility/guard/event/lineage sequence, explicit numerical and performance kill criteria (especially pp. 3–15).	Architectural basis
KC Two Hands 3.0	Verified proposals, deterministic commit, persistent constraints versus discrete events, state hashes, checkpoints and replay (pp. 11–16).	Production discipline
UGTS-KC 3.6	Literal content-addressed definitions, resolvable dependencies, canonical hashing and normative operation order (pp. 1–14).	Referential discipline
UGTS-KC 3.6.2 SCLP	Finite packed addresses, radix refinement, explicit guard intervals and finite grammar budgets (pp. 1–15).	Packing discipline
UGTS-KC 3.9 KC Elizabeth	Deterministic game world, snapshots/hashes, project validation, CLI and self-contained HTML delivery (pp. 2–16).	Game-runtime basis
UGTS-GN 1.1	Compact state/event records, pruning metrics, memory/compression contracts and strict physical-performance boundary (pp. 2–15).	Performance discipline
UGTS-KC 3.6.3 SARA	Wallet-specific cryptographic public-certificate profile.	Excluded as unrelated

1.1 Source-derived versus engineering-derived

Class	Included here	Examples
Source-derived architecture	Terms and invariant order retained from the PDFs	typed state, support, compatibility, guard, verified event, transition, lineage, content hash, replay
Engineering-derived Go layer	New rules and algorithms required by the application	board graph, groups, liberties, captures, superko, area score, minimax, D4 canonicalization, MCTS, SGF
Measured package evidence	Outputs regenerated from this package	52 tests, solved fixtures, certificate rerun, MCTS seed run, wheel, file hashes
Explicit non-claims	Results not established by the package	unrestricted 19x19 solution, production engine strength, physical-GPU advantage, universal rule-set equivalence

2. Formal Go State and Board Topology

For board size n, the board is a finite graph $G = (V, E)$ with $|V| = n^2$ and orthogonal adjacency. Every vertex contains exactly one value from $\{\emptyset, B, W\}$. The authoritative state is

$$q_{go} = (R, B, p, c, m, H, B_{prev}, L),$$

where R is the rules profile, B the board, p the player to move, c the consecutive-pass count, m the move number, H the exact superko history, B_{prev} the simple-ko board, and L the ordered replay lineage.

EXACTNESS CONDITION

Coordinates are presentation labels, not identity. A repeated coordinate can represent a legal capture cycle only when the complete resulting board position is new under the selected repetition profile. Exact state identity therefore includes the rules, side to move, pass state and repetition state.

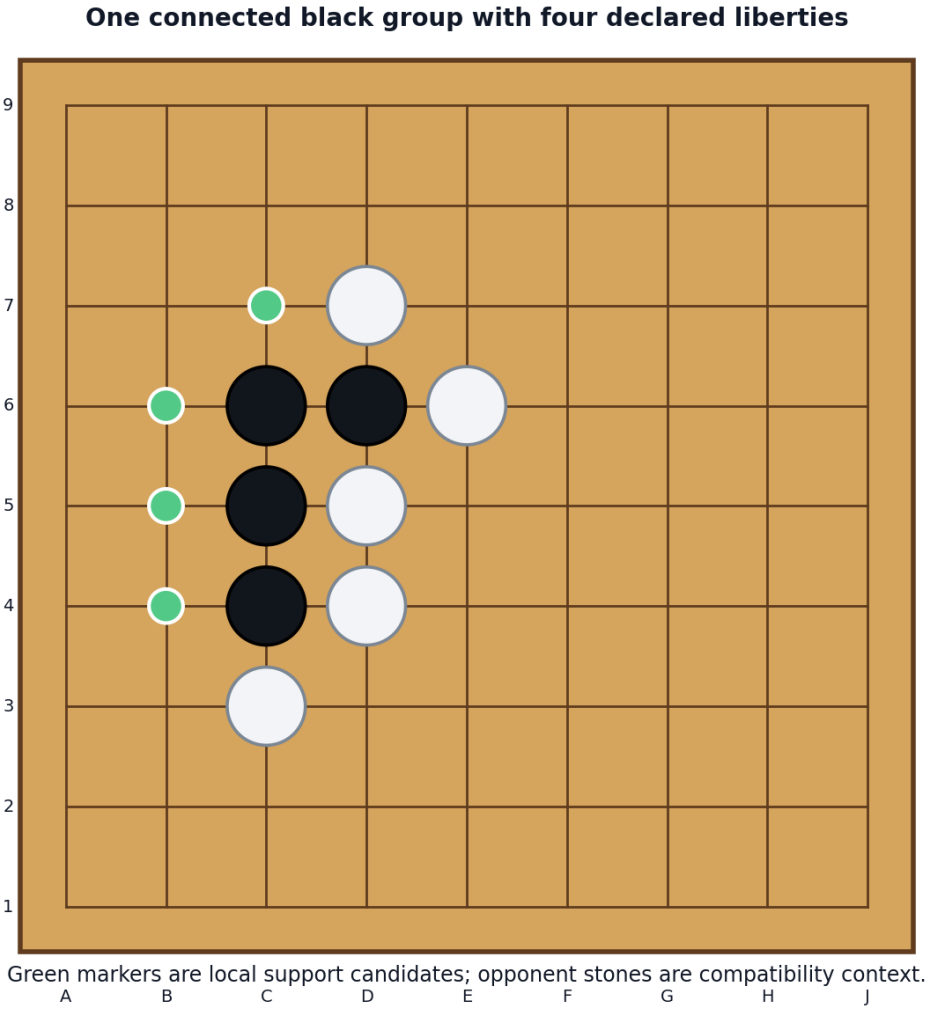


Figure 1: Groups and liberties are finite graph relations. Green points are the black group's liberty set; neighboring white stones are compatibility context rather than part of the group.

2.1 Group and liberty calculus

A group is a maximal same-color connected component. Its liberty set is the union of adjacent empty vertices. The implementation uses exact flood traversal over the precomputed neighbor table. This makes capture classification finite and deterministic:

```
place stone
for each adjacent opponent group:
    if liberties(group) == empty: remove group
if liberties(own_group) == empty:
    reject as suicide, or remove under an explicit suicide profile
apply repetition guard
commit atomically
```

2.2 Compact exact board identity

The board is also encoded as a collision-free base-3 integer

$$K_B = \sum_{i=0}^{n^2-1} B_i 3^i,$$

with empty = 0, black = 1 and white = 2. This is finite packing, not lossy compression. It is used for exact repetition sets and can reconstruct the full board.

3. Move Proposal, Guard and Atomic Commit

A move is not accepted merely because a user clicked an empty-looking point. It becomes a verified event only after the declared guard sequence.

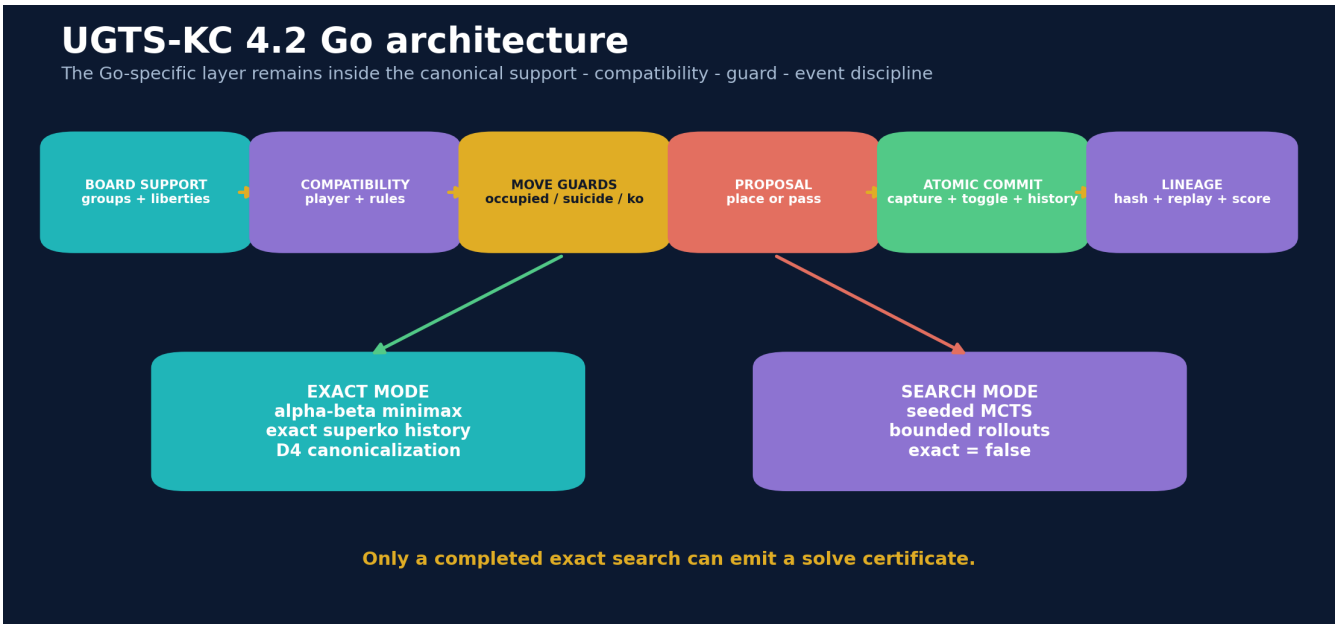


Figure 2: Go-specific mechanics mapped into the canonical UGTS event sequence.

For a non-pass proposal a:

$$\text{verified}(a, q) = \text{inboard} \wedge \text{empty} \wedge \text{capture\_defined} \wedge \text{suicide\_ok} \wedge \text{repetition\_ok}.$$

A successful commit writes a new immutable state and a move event containing color, point/pass, captured stones, optional self-removal, guard status, pre/post hashes and a lineage label.

Table 3: Move-event fields

Field	Meaning
move / color	Proposed point or pass and acting player
captured / self-removed	Exact ordered point sets changed by the rule patch
pre-hash / post-hash	SHA-256 of canonical state records including repetition history
guard status	move-accepted or pass-accepted; failures carry reason codes as exceptions
lineage label	Stable move number, color and point/pass marker for replay inspection

4. Ko, Superko and Finite Search

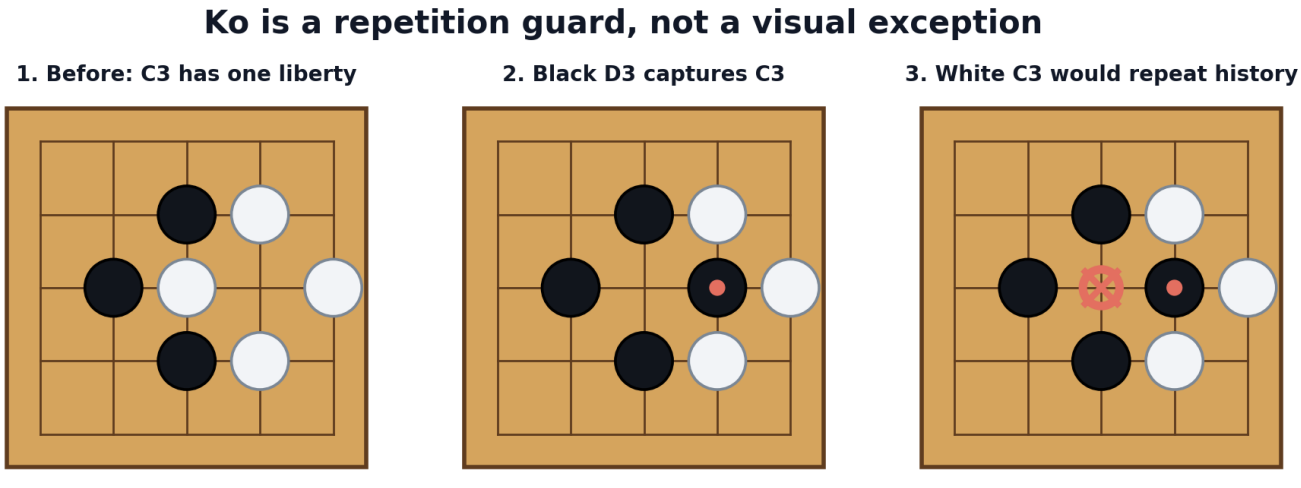


Figure 3: A positional-superko rejection is a relation over history, not a special-case pixel rule.

Profile	Guard	Exact-solver admission
Positional superko	Resulting board code has not appeared before	Yes
Situational superko	Resulting board plus next player has not appeared before	Yes
Simple ko	Resulting board differs from one ply earlier	Rules engine only
No ko	No repetition restriction	Rules engine only

The exact solver refuses simple-ko and no-ko profiles. That refusal is not a missing feature disguised as a result: those profiles can admit longer cycles, so a finite exhaustive graph requires an additional cycle policy. Positional and situational superko add a new exact key after every legal non-pass move, which bounds the game length by the finite key space plus passes.

5. Area Scoring and Terminality

Two consecutive passes create the terminal event. Area scoring then counts stones plus single-color empty regions. Mixed or unbordered regions remain neutral. Komi is stored in integer half-points:

$$M_B = 2(A_B - A_W) - K_{1/2}.$$

$M_B > 0$ is a black win, $M_B < 0$ is a white win, and $M_B = 0$ is a draw. This avoids floating-point score drift after save/load or certificate serialization.

EVIDENCE / CLAIM BOUNDARY

The exact core implements area scoring only. Japanese-style territory scoring, dead-stone agreement, seki adjudication, handicap conventions and tournament-specific edge cases require separate versioned adapters. They are not silently merged into this profile.

6. Exact Solver Architecture

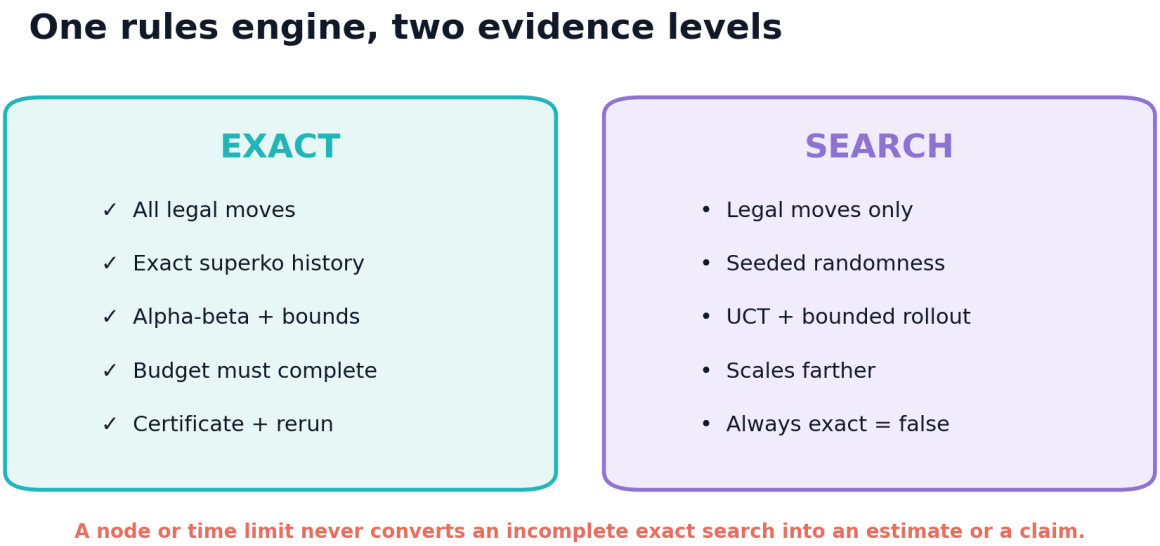


Figure 4: Exact and heuristic modes share legal transitions but have different evidence status.

6.1 Minimax and alpha-beta

Black maximizes and White minimizes the exact terminal half-point margin. The solver uses alpha-beta pruning and a transposition table with exact, lower and upper bounds. Move ordering considers terminal passes, capture count, a non-authoritative intermediate area heuristic and point index. Ordering changes speed, not completeness.

6.2 History-correct transposition identity

A transposition entry is keyed by

$$(B, p, c, B_{prev}, H).$$

Under optional D4 canonicalization, the current board, previous board and *every* history code are transformed by the same rotation/reflection before the minimum representative is selected. Transforming only the current board would be incorrect because it could alter which future moves violate superko.

6.3 Budget semantics

Node and monotonic-time limits are first-class guards. If either limit is crossed, the solver returns `complete=false`, no minimax value, no best move and no certificate. The captured empty-3x3 run stopped at 2049 nodes (node limit 2048 exceeded) and correctly emitted no exact value.

```
result = ExactSolver(node_limit=2048).solve(empty_3x3)
assert result.complete is False
assert result.value_halfpoints is None
# No solve certificate is permitted.
```

7. Content-Addressed Solve Certificate

The certificate follows the 3.6 referential discipline: canonical JSON, explicit schema, stable IDs and SHA-256 content identity. It contains the full root state, exact result, best move, principal variation, solver mode, metrics, D4 setting, transposition-table digest and a claim boundary.

Verification occurs in two stages:

1. Remove the certificate hash field, canonicalize the remaining JSON and recompute SHA-256.

2. Reconstruct the root state, rerun exhaustive search and compare exact value plus transposition digest.

Table 4: Verified 2x2 certificate identifiers

Identifier	Value
Root state SHA-256	`1a3896fc9a9474e851babc5930873284d4f793aba85e08a94b13ef3127ad4294`
Certificate SHA-256	`eaec58fd1dbf2ccd0b81981750385b5112b858229f599c0baaa0c4fd17f4be0d`
Transposition digest	`39bcfe46fdb8702fec10a4900fe0b22d8cdf8301dbd223d0c5c7ca1f72235f13`
Search nodes / terminal nodes	742 / 196
Table entries / cutoffs	538 / 343
Maximum ply	38
Rerun status	verified by deterministic exhaustive rerun

EVIDENCE / CLAIM BOUNDARY

The certificate is executable evidence from this implementation. It is stronger than a screenshot or unsupported result string, but it is not a proof-assistant theorem and does not independently certify the Python interpreter, hardware or compiler.

8. Solved Fixtures

Table 5: Exact initial-board fixture results

Board	Komi	Winner	Black margin	Nodes	TT entries	PV prefix
1x1	0.0	draw	0.0	4	2	pass, pass
1x1	0.5	white	-0.5	4	2	pass, pass
2x2	0.0	black	1.0	742	538	A2, B1, B2, A1, B2
2x2	0.5	black	0.5	742	538	A2, B1, B2, A1, B2
2x2	1.0	draw	0.0	742	538	A2, B1, B2, A1, B2
2x2	1.5	white	-0.5	742	538	A2, B1, B2, A1, B2

The empty 2x2 result is invariant under D4: any corner is equivalent as the first move. Deterministic tie order selects A2. The full principal variation is:

```
A2, B1, B2, A1, B2, A2, B2, A1, A2, pass, pass
```

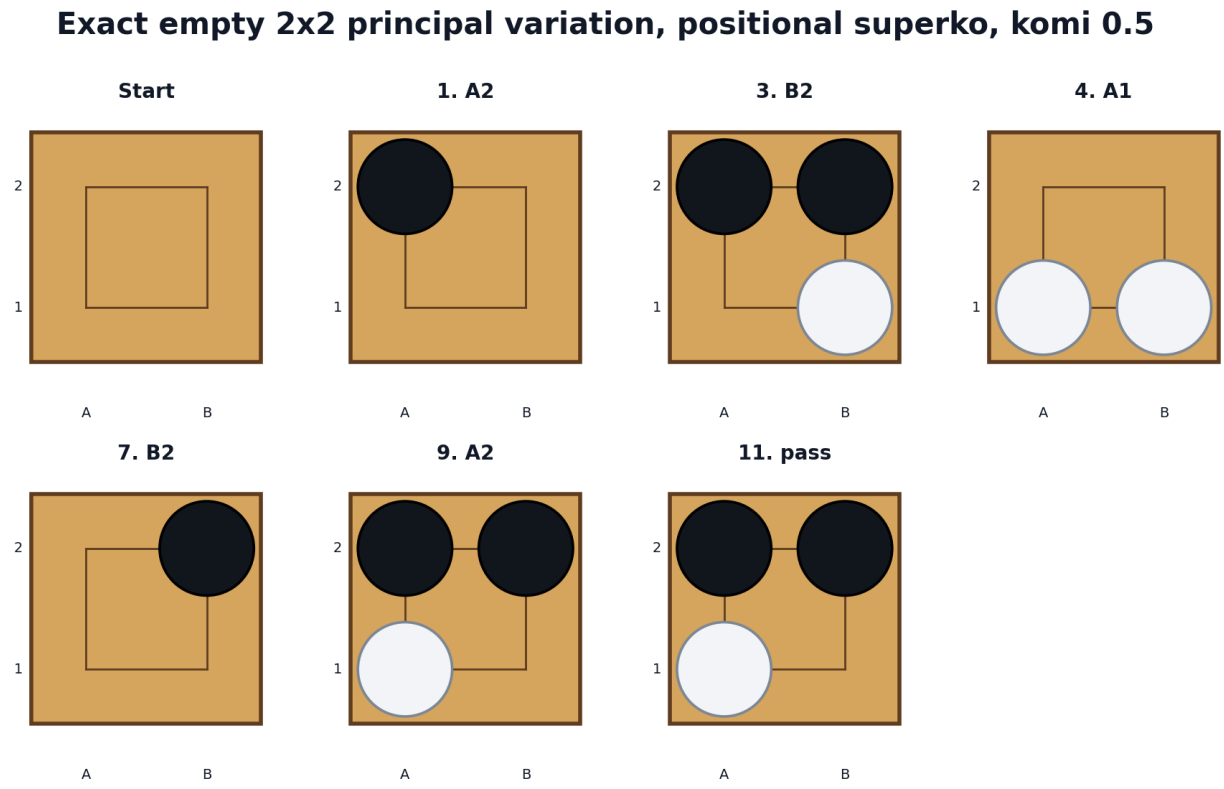


Figure 5: Selected states from the exact empty-2x2 principal variation.

The 1x1 result is a draw at zero komi and White by 0.5 at 0.5 komi. The 2x2 threshold under this profile is Black +1.0 before komi: zero komi gives Black +1.0, 1.0 komi gives a draw, and 1.5 komi gives White +0.5.

9. Scalable Search for Larger Boards

Exact history makes the reference solver intentionally conservative. For larger boards, the package supplies seeded MCTS using the same legal transition engine. Tree expansion never bypasses occupancy, capture, suicide or superko guards. Rollouts probe a bounded random subset of empty points and always report `exact=false`.

The captured 9x9 run used 100 iterations, rollout limit 128 and seed 420042. It completed in 0.672 seconds on the validation environment and selected F4; the estimated black win rate was 0.350. These values are reproducibility evidence, not playing-strength or solution claims.

Table 6: Top seeded 9x9 root children by visits

Move	Visits	Estimated black win rate
F9	2	0.500
G9	2	0.500
E8	2	1.000
F8	2	0.500
D7	2	0.500
J7	2	0.500
C6	2	0.500
F6	2	1.000
J6	2	0.500
B5	2	0.500

9.1 Path toward proof-scale Go

A credible exact 19x19 program would require additive mechanisms beyond this reference implementation:

- proof-number or retrograde search specialized to win/loss thresholds;
- mathematically safe decomposition of independent endgames and local life/death regions;
- seki, ko-threat and global-temperature certificates that preserve cross-region coupling;
- distributed content-addressed proof shards with deterministic merge rules;
- verified transposition storage larger than RAM, with collision-free state reconstruction;
- GPU kernels restricted to candidate generation or bound computation, while authoritative commit and proof checks remain deterministic;
- independent verifier implementations and cross-platform differential testing.

None of these is silently claimed as complete here.

10. Storage, Compression and Reconstructibility

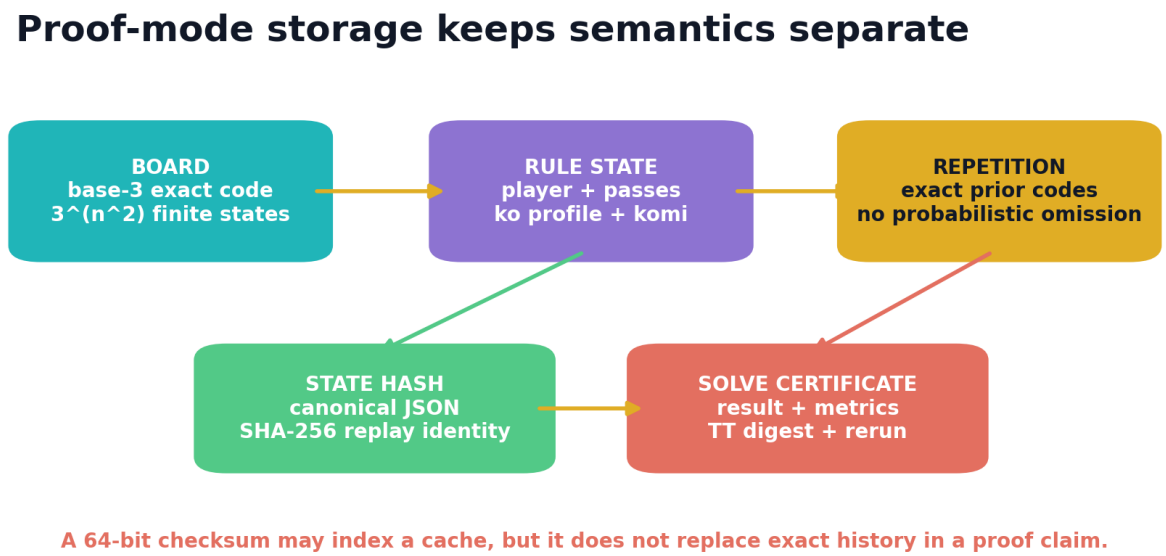


Figure 6: Compact board identity does not erase rules state or repetition history.

Base-3 board codes are exact and compact. A situational key packs the board code and player into one integer. SHA-256 hashes are used for content and replay integrity. However, proof mode retains the exact history set. A 64-bit checksum may index a cache, but it cannot replace exact history without a collision policy and independent reconstruction evidence.

The strongest compression opportunity is therefore not blind state truncation; it is structure:

- reuse immutable board records and symmetry classes;
- store canonical rules once and reference them;
- compact verified event logs when the initial state and transition function reconstruct every state;
- shard exact histories by content address;
- preserve full exceptions whenever external input, rule change or unresolved ambiguity breaks reconstruction.

11. Command-Line, SGF and Browser Delivery

The CLI exposes exact solve, seeded MCTS, score, web build and certificate verification:

```
PYTHONPATH=src python -m ugts_go solve \
  --size 2 --komi 0.5 \
  --certificate validation/solve_2x2_k0_5.json
```

```
PYTHONPATH=src python -m ugts_go verify-certificate \
  validation/solve_2x2_k0_5.json

PYTHONPATH=src python -m ugts_go mcts \
  --size 9 --komi 7.5 --iterations 200 --seed 420042

PYTHONPATH=src python -m ugts_go build-web examples/web/go_ugts.html
```

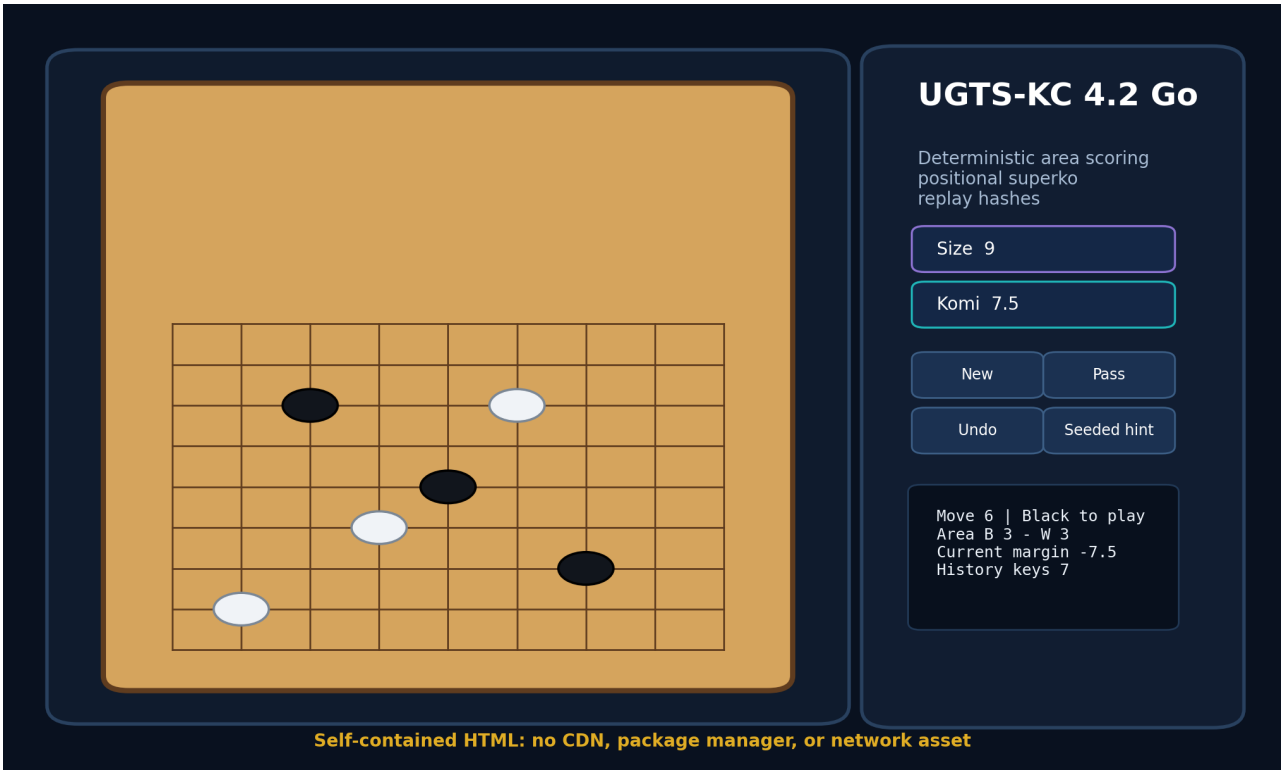


Figure 7: The included browser board is a single HTML file with local rules, score, pass, undo and a bounded seeded hint.

The SGF adapter supports one variation with size, komi, black/white moves and passes. It is intentionally a downstream interchange subset, not a full SGF property engine or authoritative substitute for the serialized UGTS state.

12. Validation and Release Evidence

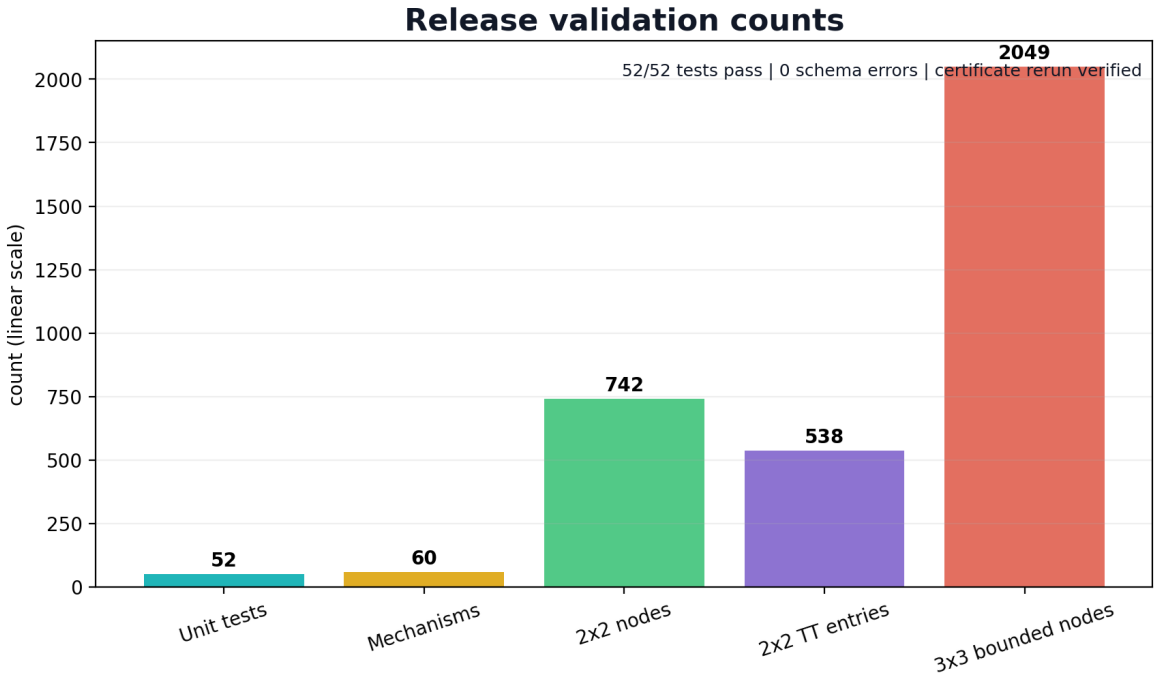


Figure 8: Measured release counts. The empty-3x3 bar is a bounded interrupted attempt, not a solved result.

Gate	Result	Established boundary
Python compile	PASS	All source modules compile under Python 3.13.5.
Unit tests	52/52 PASS	Rules, topology, packing, solver, certificates, MCTS, SGF and web builder satisfy included properties.
JSON Schema	0 errors	Example state and exact certificate satisfy the supplied Draft 2020-12 schema.
Mechanism catalog	60 contiguous	M390–M449 count and order machine-checked.
Exact certificate	PASS	Hash and deterministic rerun match.
Web output	6005 bytes	Contains Canvas runtime and no external HTTP resource.
Wheel build	PASS	Dependency-free wheel built and smoke-tested with build isolation disabled.
Large-board strength	NOT CLAIMED	Seeded MCTS evidence is not a benchmark against established Go engines.
19x19 exact solution	NOT CLAIMED	No root certificate exists in this release.

13. Kill Criteria and Explicit Non-Claims

The profile must be rejected, simplified or relabeled if any of the following occurs:

- rules, komi, suicide, ko or scoring conventions are implicit;
- superko history is omitted while claiming exact legality;
- a heuristic move filter is used in proof mode;
- D4 canonicalization transforms the board but not all repetition state;
- floating-point score spelling changes state identity;
- an incomplete search emits a best move as exact or creates a certificate;
- MCTS estimates are described as solved values;
- cache hashes replace full proof state without collision/reconstruction evidence;

- GPU or distributed workers mutate authoritative state without verified deterministic commit;
- a conventional Go engine is cheaper, faster or more accurate for the target requirement.

EVIDENCE / CLAIM BOUNDARY

No universal O(1) game solver, zero-memory proof, infinite compression, one-bit complete state, perfect 19x19 play, native physical-GPU advantage, Japanese-rules equivalence, multiplayer service, tournament certification or legal ownership claim is introduced.

14. Final Definition

FORMAL DECISION

UGTS-KC 4.2 Go is a finite, typed, event-authoritative application profile in which an orthogonal board graph supplies groups and liberties; rules compatibility selects player, ko, suicide and scoring conventions; occupancy, liberty and repetition become guards; a legal placement or pass becomes a verified proposal; capture, side toggle and history update commit atomically; terminal area score is represented in exact half-points; exact minimax retains complete superko state and may emit a content-addressed rerunnable certificate only after exhaustive completion; seeded MCTS remains a separately typed estimate; and SGF/browser output stays downstream of authoritative state and lineage.

Canonical shorthand:

```
board/group support -> rules compatibility -> move guards -> verified proposal -> atomic commit
                                -> hash/replay -> score/certificate
```

The release is technically coherent to the extent demonstrated by the bundled implementation and validation. The next research target is not to rename heuristic strength as a solution, but to add independently verifiable decomposition and proof-search machinery until larger roots can emit complete certificates.

A. Mechanism Catalog M390–M449

The following 60 entries are engineering additions. They extend the combined catalog after M389 but do not rewrite the source-normalized or prior KC mechanisms.

Table 7: Complete UGTS-KC 4.2 Go delta catalog

ID	Domain	Mechanism	Normalized technical definition	Validation
M390	Board topology	Finite orthogonal board graph	Represent an n x n Go board as a finite vertex set with four-neighbor adjacency.	Implemented + tested
M391	Board topology	Typed intersection state	Each vertex stores empty, black, or white as a closed three-value type.	Implemented + tested
M392	Board topology	Go coordinate chart	Map indices to letter-number coordinates while skipping I and retaining row orientation.	Implemented + tested
M393	Board topology	Connected color group	Compute maximal same-color connected components under orthogonal adjacency.	Implemented + tested
M394	Board topology	Liberty relation	A group liberty is an adjacent empty vertex; liberty sets are exact finite relations.	Implemented + tested
M395	Board topology	Empty-region decomposition	Partition empty vertices into connected regions and record bordering colors.	Implemented + tested
M396	Board topology	Immutable board record	Store the board as immutable bytes and create a new record on commit.	Implemented + tested
M397	Board topology	Base-3 exact board code	Encode all intersections in a collision-free base-3 integer.	Implemented + tested
M398	Move legality	Occupied-point guard	Reject placement when the target intersection is not empty.	Implemented + tested
M399	Move legality	Opponent capture resolution	After placement, remove every adjacent opponent group with zero liberties.	Implemented + tested
M400	Move legality	Self-liberty guard	After captures, require the placed stone's group to have a liberty unless suicide is enabled.	Implemented + tested
M401	Move legality	Optional suicide removal	When enabled, remove the zero-liberty friendly group as an explicit rule profile.	Implemented + tested
M402	Move legality	Simple-ko previous-board guard	Reject a move reproducing the board from one ply earlier.	Implemented + tested
M403	Move legality	Positional-superko guard	Reject a non-pass move whose exact board code is already in history.	Implemented + tested
M404	Move legality	Situational-superko guard	Reject a non-pass move whose board and next-player code is already in history.	Implemented + tested
M405	Move legality	Pass exception and side toggle	A pass changes side and pass count without occupancy or repetition rejection.	Implemented + tested
M406	Scoring/end	Two-pass terminal event	Two consecutive verified passes produce game terminality.	Implemented + tested
M407	Scoring/end	Stone area count	Each on-board stone contributes one area point to its color.	Implemented + tested
M408	Scoring/end	Single-color empty-region territory	Assign an empty region only when its bordering color set contains exactly one color.	Implemented + tested
M409	Scoring/end	Neutral-region classification	Mixed or unbordered empty regions are explicitly neutral.	Implemented + tested
M410	Scoring/end	Half-point komi arithmetic	Store komi and score margins in integer half-points.	Implemented + tested
M411	Scoring/end	Winner relation	Positive black margin means black, negative means white, zero means draw.	Implemented + tested
M412	Scoring/end	Terminal score query	Score is authoritative only after terminality in exact game solving.	Implemented + tested
M413	Scoring/end	Area-only core boundary	The exact core implements area scoring; territory/dead-stone adjudication is an adapter target.	Boundary enforced
M414	Exact search	Finite-history minimax state	Search keys include board, player, pass state, previous board, and exact superko history.	Implemented + tested
M415	Exact search	Alpha-beta minimax	Black maximizes and white minimizes exact terminal half-point margin.	Implemented + tested
M416	Exact search	Deterministic move ordering	Order terminal moves, captures, heuristic margins, and point indices deterministically.	Implemented + tested
M417	Exact search	Transposition bound types	Store exact, lower, and upper values under standard alpha-beta semantics.	Implemented + tested
M418	Exact search	Node budget guard	Abort exact mode when the declared node limit is exceeded.	Implemented + tested
M419	Exact search	Time budget guard	Abort exact mode when the monotonic time budget is exceeded.	Implemented + tested
M420	Exact search	Principal variation reconstruction	Choose deterministic children matching the exact root value until terminality or a declared limit.	Implemented + tested
M421	Exact search	Per-move exact analysis	Evaluate every legal child with exact minimax after a completed solve.	Implemented
M422	Exact search	Superko-only exact admission	Refuse simple-ko and no-ko profiles in proof mode because finiteness is not guaranteed by the profile.	Implemented + tested
M423	Exact search	Exact-versus-estimate type split	SolveResult.complete and MCTSResult.exact prevent heuristic output from masquerading as proof.	Implemented + tested
M424	Symmetry/packing	D4 board transform family	Provide identity, rotations, and reflections as eight exact index maps.	Implemented + tested
M425	Symmetry/packing	History-correct symmetry	Transform current board, previous board, and every superko key under one shared symmetry.	Implemented + tested
M426	Symmetry/packing	Lexicographic canonical state	Select the minimal transformed state/history tuple as the transposition representative.	Implemented + tested
M427	Symmetry/packing	Exact transform inverse	Every D4 transform has a tested inverse.	Implemented + tested
M428	Symmetry/packing	Compact situational code	Pack board code and player into one collision-free integer for situational superko.	Implemented + tested
M429	Symmetry/packing	Canonical state SHA-256	Hash sorted canonical JSON including rules and repetition history.	Implemented + tested
M430	Symmetry/packing	Transposition-table digest	Hash sorted canonical entry records containing key, value, and bound.	Implemented + tested
M431	Symmetry/packing	Memory boundary	Exact history is retained even when it dominates memory; probabilistic hashes are not substituted in proof mode.	Boundary enforced
M432	Heuristic search	Seeded MCTS root	Use a declared pseudorandom seed for reproducible search estimates.	Implemented + tested
M433	Heuristic search	UCT selection	Balance exploitation and exploration by visit-weighted upper confidence.	Implemented

ID	Domain	Mechanism	Normalized technical definition	Validation
M434	Heuristic search	Legal expansion	Expand only transitions accepted by the authoritative rules engine.	Implemented + tested
M435	Heuristic search	Bounded rollout	Stop random playout after terminality or a declared ply budget.	Implemented
M436	Heuristic search	Area-evaluation rollout result	Convert rollout area margin to black win, white win, or draw value.	Implemented
M437	Heuristic search	Visit-count move selection	Choose the most visited root child with deterministic tie behavior.	Implemented + tested
M438	Heuristic search	Child statistics report	Return move, visits, and estimated black win rate.	Implemented
M439	Heuristic search	Non-proof marker	Every MCTS result records exact=false.	Implemented + tested
M440	Project/runtime	Versioned rules record	Serialize size, komi, ko, suicide, and scoring as explicit project data.	Implemented + schema
M441	Project/runtime	Move event record	Record color, move, captures, self-removal, guard status, hashes, and lineage label.	Implemented + tested
M442	Project/runtime	Replay executor	Apply an ordered move list and return final state plus ordered events.	Implemented + tested
M443	Project/runtime	Solve certificate	Serialize root state, exact result, metrics, table digest, and content hash.	Implemented + tested
M444	Project/runtime	Certificate hash verifier	Recompute canonical JSON SHA-256 after removing the certificate hash field.	Implemented + tested
M445	Project/runtime	Deterministic rerun verifier	Re-solve the root state and compare value plus table digest.	Implemented + tested
M446	Project/runtime	SGF single-variation adapter	Import/export board size, komi, moves, and passes for one SGF variation.	Implemented + tested
M447	Project/runtime	Self-contained browser board	Generate one HTML file with board, legal moves, score, undo, pass, and seeded hint.	Implemented + tested
M448	Project/runtime	Command-line workflow	Expose solve, MCTS, score, web build, and certificate verification commands.	Implemented
M449	Project/runtime	Validation and manifest gate	Compile, test, solve fixtures, validate certificate, build package, and hash final files.	Implemented in validation

B. Package Map

Path	Contents
`src/ugts_go/`	Immutable model, board topology, rules, hashes, D4 symmetries, exact solver, MCTS, certificates, SGF, web builder and CLI.
`spec/`	Formal definition, JSON Schema, source-integration register and M390–M449 CSV/JSON catalog.
`tests/`	52 dependency-free tests.
`examples/`	Small-board exact solve, ko guard, 9x9 MCTS and self-contained HTML.
`validation/`	Captured test/compile/schema results, solved fixtures, exact certificate, bounded 3x3 attempt, MCTS evidence and package checks.
`dist/`	Installable Python wheel.
`checksums/`	Complete SHA-256 manifest for distributable files.
`report/`	This PDF, LaTeX source and report generation assets.

C. Reproduction

From the package root:

```
PYTHONPATH=src python tools/build_catalog.py
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python tools/run_validation.py
PYTHONPATH=src python examples/solve_small_boards.py
python -m pip wheel . --no-deps --no-build-isolation -w dist
```

Report assets and PDF:

```
PYTHONPATH=src python tools/generate_report_assets.py
python tools/generate_report_tex.py
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  report/UGTS_KC_4_2_GO_Solver.tex
```

D. Attribution and Evidence Notice

Prepared for Tom Klootwijk as an engineering artifact within the UGTS project. Requester attribution is recorded as supplied and remains independently unverified. This report and package are not legal proof of identity, authorship, ownership, patentability, priority, exclusive rights, licensing status or chain of title. The source PDFs are referenced by filename and technical role; they are not redistributed inside this package.